

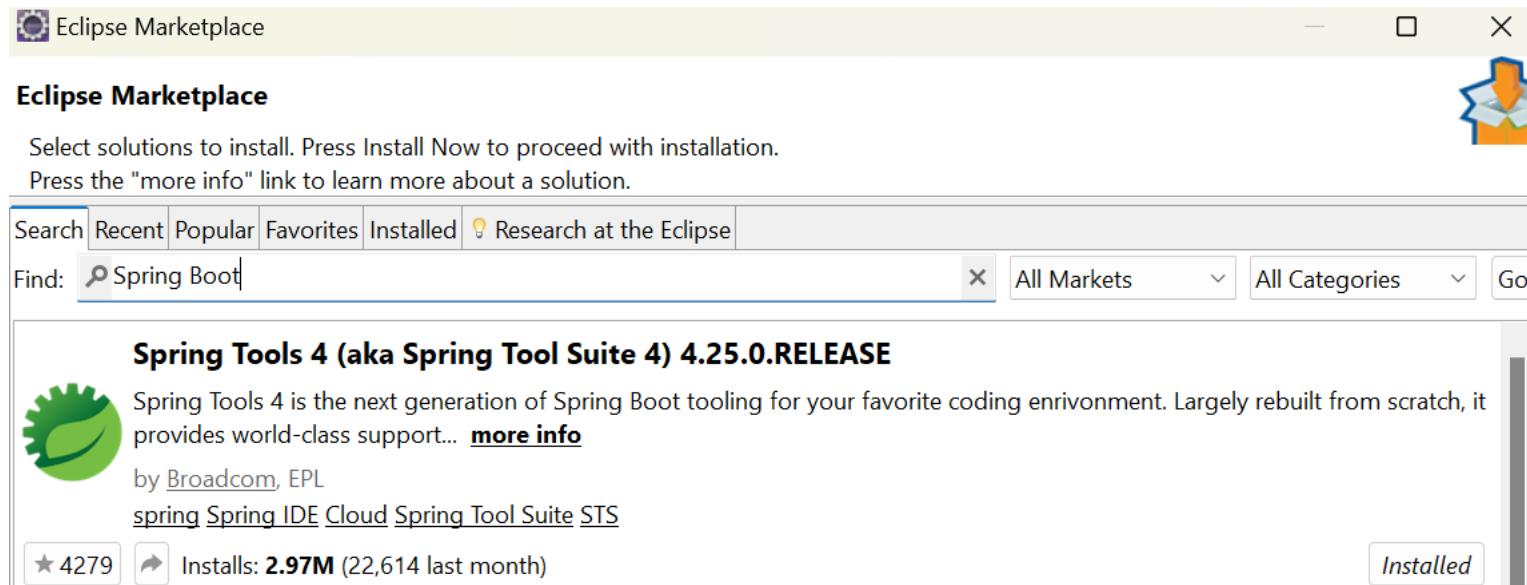
Chương 4: Lập trình ứng dụng web với Spring Framework

Khái niệm

- ▶ **Spring Framework** là một framework mã nguồn mở được phát triển dựa trên nền tảng Java, cung cấp mô hình lập trình và cấu hình cho các ứng dụng doanh nghiệp. Hệ sinh thái Spring bao gồm nhiều module trong đó có Spring Boot và Spring MVC.
- ▶ **Spring Boot**: là một module nằm trong Spring Framework, cung cấp giao diện và khả năng phát triển các ứng dụng độc lập một cách đơn giản.
- ▶ **Spring MVC**: hỗ trợ phát triển các ứng dụng web dựa trên mô hình MVC

Cài đặt Spring boot từ eclipse

- ▶ Trên eclipse, chọn menu Help -> Eclipse Marketplace
- ▶ Gõ từ khóa tìm kiếm “Spring Boot”, chọn cài đặt Spring Tool 4
- ▶ Thực hiện theo các bước hướng dẫn cài đặt



Tạo 1 project Spring boot từ trang start.spring.io

- ▶ Mở trình duyệt web, truy cập trang start.spring.io
- ▶ Chọn các tùy chọn cho project:
 - ▶ Project: Maven
 - ▶ Language: Java
 - ▶ Spring Boot: 3.3.4
- ▶ Project Metadata:
 - ▶ Group: tên đơn vị, thường ghi ngược tên miền
 - ▶ Artifact: tên dự án
 - ▶ Name: tên dự án
 - ▶ Description: Miêu tả dự án
 - ▶ Package name: Tên gói
 - ▶ Packaging: jar
 - ▶ Java: 23



Project

- ☐ Gradle - Groovy
- ☐ Gradle - Kotlin
- ☒ Maven

Language

- ☒ Java ☐ Kotlin
- ☐ Groovy

Spring Boot

- ☐ 3.4.0 (SNAPSHOT) ☐ 3.4.0 (M3)
- ☐ 3.3.5 (SNAPSHOT) ☒ 3.3.4 ☐ 3.2.11 (SNAPSHOT)
- ☐ 3.2.10

Project Metadata

Group com.example

Dependencies

ADD DEPENDENCIES... CTRL + B

Spring Web WEB

Build web, including RESTful, applications using Spring MVC.
Uses Apache Tomcat as the default embedded container.

MySQL Driver SQL

MySQL JDBC driver.



GENERATE CTRL + G

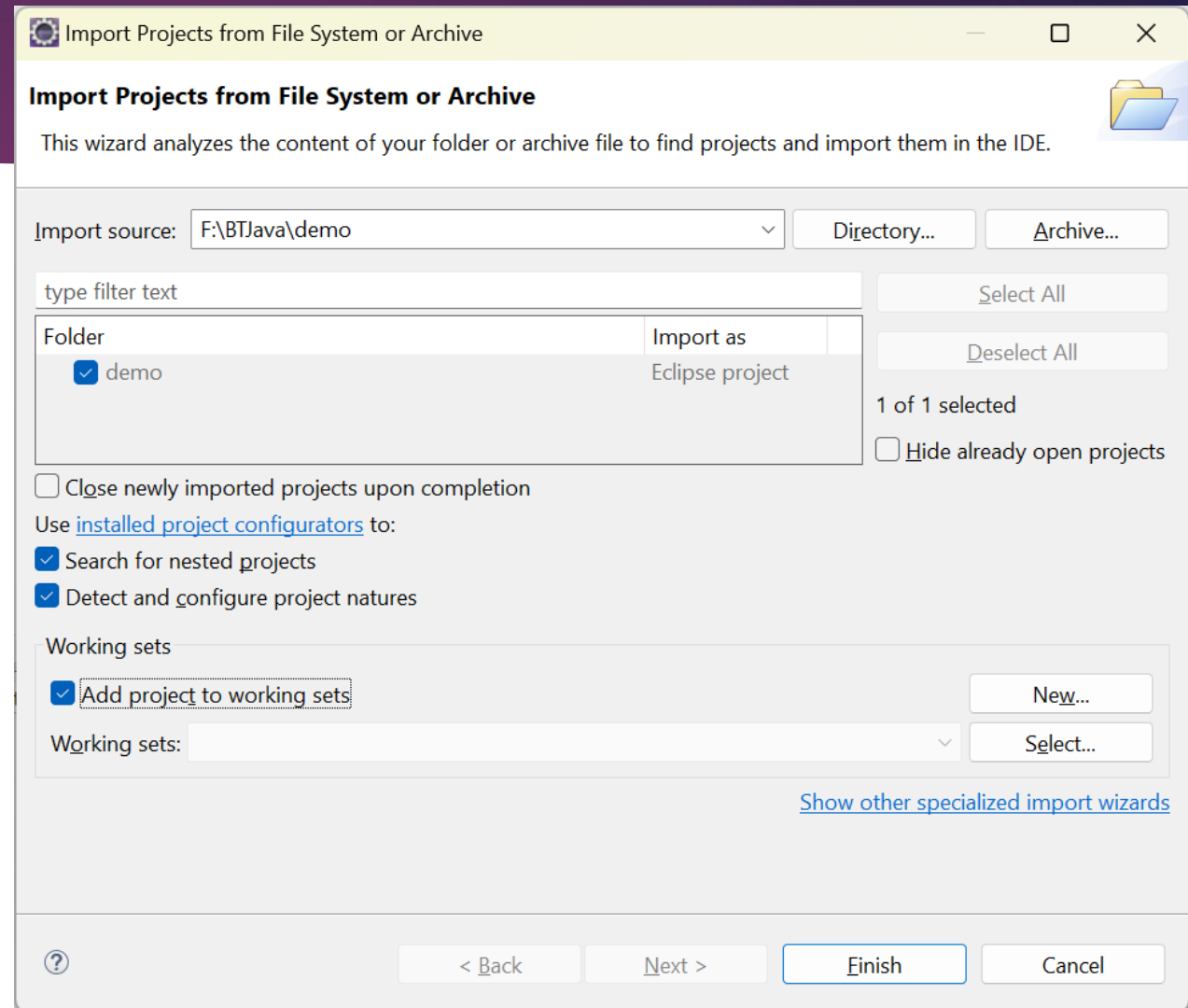
EXPLORE CTRL + SPACE

SHARE...

► Nhấp nút GENERATE để sinh project và tự động tải về. Giải nén và mở lên từ Eclipse

Mở một Project Spring Boot từ Eclipse

- ▶ File -> Open Projects from File System.
- ▶ Nhấp nút Directory để chọn thư mục Project cần mở
- ▶ Chọn Add project to working sets



Annotation

- ▶ **Spring Annotations** là một dạng metadata cung cấp dữ liệu về một chương trình. Chú thích được sử dụng để cung cấp thông tin bổ sung về một chương trình.
- ▶ **@Controller** : Chú thích @Controller chỉ ra rằng một lớp cụ thể phục vụ vai trò của bộ điều khiển.
- ▶ **@RestController**: chú thích mới, là sự kết hợp của @Controller và @ResponseBody để đơn giản hóa việc tạo ra các Restful web service.

Ví dụ về @Controller

@Controller

```
@RequestMapping("books")
```

```
public class SimpleBookController {  
    @GetMapping("/{id}", produces = "application/json")  
    public @ResponseBody Book getBook(@PathVariable int id)  
    {  
        return findBookById(id);  
    }  
    private Book findBookById(int id) { // ... }  
}
```


Ví dụ về @RestController

@RestController

@RequestMapping("books-rest")

```
public class SimpleBookRestController {  
    @GetMapping("/{id}", produces =  
    "application/json")  
    public Book getBook(@PathVariable int id) {  
        return findBookById(id);  
    }  
    private Book findBookById(int id) { // ...  
}
```

@Autowire

- ▶ Tự động nhúng các bean được Spring Container sinh ra vào các class được khai báo **@Autowire** (theo cơ chế **Dependency Injection**)

```
//Properties
@Service
public class UserService {
    @Autowired
    UserRepository userRepository;
    @Autowired
    FacebookUtil facebookUtil;
}
```

```
//Setter
@Service
public class User {
    Private String name;
    @Autowired
    void setName(String name) {
        this.name = name;
    }
}
```



```
//Constructor
```

```
@Service
```

```
public class User {
```

```
    private long id;
```

```
    private String name;
```

```
    @Autowired void user(long id, String name) {
```

```
        this.id = id;
```

```
        this.name = name;
```

```
    }
```

```
}
```

@Configuration

- ▶ Được sử dụng để chỉ ra rằng, Class khai báo sử dụng **@Configuration** sẽ khai báo một hoặc nhiều **@Bean** method trong class đó.
→ Thông thường các Bean cấu hình trong dự án ta sẽ để trong các lớp configuration này. Ví dụ cấu hình Elasticsearch, Thymeleaf, đa ngôn ngữ,...

@Bean

- ▶ Đánh dấu trên method thông báo cho Spring, method đó sẽ sinh ra một bean và được quản lí bởi **Spring Container**.

→ Tất cả các Method sử dụng annotation **@Bean** phải nằm trong class Configuration.

```
@Configuration
public class WebDriverConfig {
    @Bean
    public WebDriver Chrome() {
        System.setProperty("webdriver.chrome.driver",
            "/Downloads/chromedriver_linux64/chromedriver");
        return new ChromeDriver();
    }
}
```

@Service

- ▶ Đây là annotation đặc biệt của @Component. Được dùng để xử lý nghiệp vụ, logic.

```
@Service
public class UserService {
    @Autowired
    UserRepository userRepository;
    @Autowired
    FacebookUtil facebookUtil;
}
```

@Repository

- Đây cũng là một annotation đặc biệt của @Component. Được dùng để thao tác với cơ sở dữ liệu.
- Jpa sẽ cung cấp cho các hàm select, update,... cơ bản. Có thể áp dụng thêm **Query Creation**.
- Các **Interface** thường gặp: *CrudRepository, JpaRepository, MongoRepository,...*

```
@Repository
public interface UserRepository extends JpaRepository <User, Integer> {
    //Query Creation
}
```

@ReponseBody

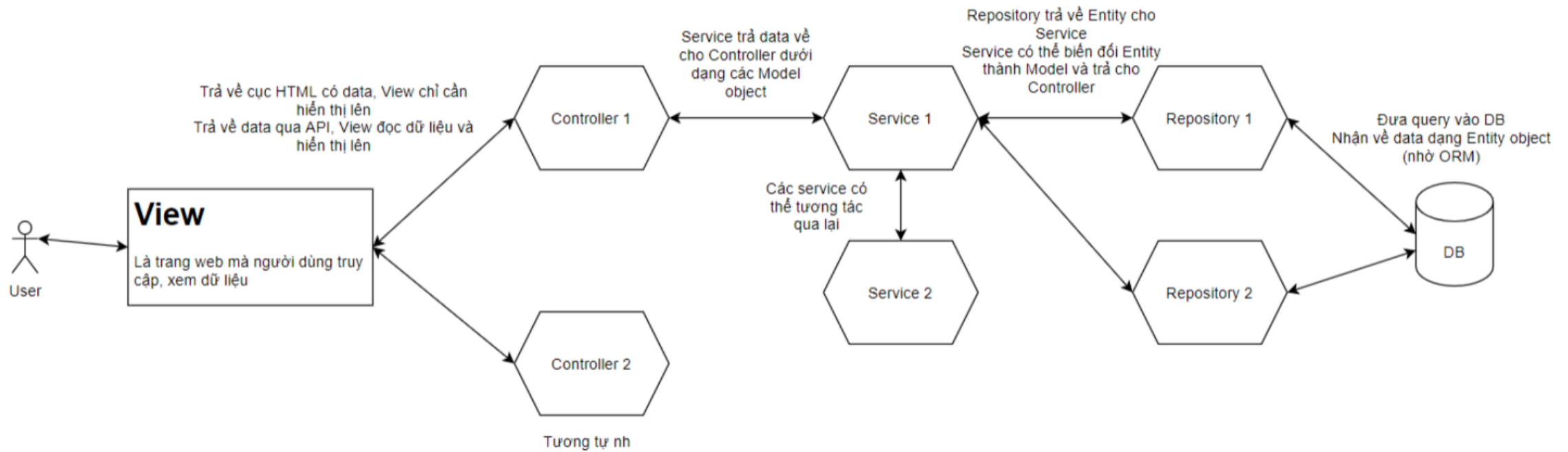
- Thông báo cho người dùng biết rằng API ở controller, sẽ trả về một đối tượng Object kiểu Json cho client chứ không render ra một trang view.

Luồng dữ liệu trong Spring Boot

Các thành phần quan trọng

- ▶ Controller: trả về View (có chứa data sẵn, dạng trang HTML), hoặc Model thể hiện dưới dạng API cho View (View viết riêng bằng React, Vue, hoặc Angular).
- ▶ Service: chứa các code tính toán, xử lý. Khi Controller yêu cầu, thì Service tương ứng sẽ tiếp nhận và cho ra dữ liệu trả cho Controller (trả về Model). Controller sẽ gửi về View như trên.
- ▶ Repository: Service còn có thể tương tác với service khác, hoặc dùng Repository để gọi Database. Repository là đối tượng trực tiếp tương tác, đọc ghi dữ liệu trong DB và trả cho service.

Sơ đồ luồng dữ liệu



ORM

- ▶ ORM (Object Relational Mapping) framework là một cơ chế cho phép người lập trình thao tác với database một cách hoàn toàn tự nhiên thông qua các đối tượng.
- ▶ ORM giúp đơn giản hoá việc tạo ra dữ liệu, thao tác dữ liệu và truy cập dữ liệu. Đó là một kỹ thuật lập trình để ánh xạ đối tượng vào dữ liệu được lưu trữ trong cơ sở dữ liệu.

Persistence layer

- ▶ Một ứng dụng có thể được chia làm 3 phần như sau: giao diện người dùng (presentation layer), phần xử lý nghiệp vụ (business layer) và phần chứa dữ liệu (data layer). Cụ thể ra, business layer có thể được chia nhỏ thành 2 layer con là business logic layer và persistence layer.
 - Business logic layer: các tính toán logic nhằm thỏa mãn yêu cầu người dùng.
 - Persistence layer: chịu trách nhiệm giao tiếp với data layer (thường là một hệ quản trị cơ sở dữ liệu quan hệ – Relational DBMS). Persistence sẽ đảm nhiệm các nhiệm vụ mở kết nối, truy xuất và lưu trữ dữ liệu vào các Relational DBMS.

Hibernate

- ▶ Hibernate là một trong những ORM Framework. Hibernate framework là một framework cho persistence layer.
- ▶ Hibernate giúp lập trình viên viết ứng dụng Java có thể map các object (POJO) với hệ quản trị cơ sở dữ liệu quan hệ (database), và hỗ trợ thực hiện các khái niệm lập trình hướng đối tượng với cơ sở dữ liệu quan hệ.
- ▶ Hibernate cho phép bạn truy vấn dữ liệu thông qua Java Persistence API (JPA) hoặc bằng ngôn ngữ SQL mở rộng của Hibernate (HQL) hoặc bằng SQL thuần (Native SQL).

Kết nối cơ sở dữ liệu MySQL

application.properties

- ▶ `spring.datasource.url=jdbc:mysql://localhost:3306/banhang`
- ▶ `spring.datasource.username=root`
- ▶ `spring.datasource.password=`
- ▶ `spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver`
- ▶ `spring.jpa.show-sql: true`
- ▶ `spring.jpa.hibernate.naming.implicit-strategy=org.hibernate.boot.model.naming.ImplicitNamingStrategyLegacyJpaImpl`
- ▶ `spring.jpa.hibernate.naming.physical-strategy=org.hibernate.boot.model.naming.PhysicalNamingStrategyStandardImpl`
- ▶ `spring.jpa.hibernate.ddl-auto=update`

Repository

```
public interface UserRepository extends  
JpaRepository<User, String> {  
  
}
```

Không cần định nghĩa thêm method, chỉ sử dụng
.findAll() có sẵn.

Service

```
@Service
public class UserService {
    @Autowired
    private UserRepository userRepository;

    public List<UserModel> getUserList() {
        List<User> users = userRepository.findAll();
        List<UserModel> userModels = new ArrayList<>();
        for (User user: users)
            userModels.add(new UserModel(user));

        return userModels;
    }
}
```

Controller

```
@RestController
@RequestMapping("/")
public class UserController {
    @Autowired
    private UserService userService;
    @GetMapping("/Nguoidung")
    public ResponseEntity<List<UserModel>>
    getUserList() {
        return new ResponseEntity<List<UserModel>>
        (userService.getUserList(),HttpStatus.ACCEPTED);
    }
}
```

Model

```
public class UserModel {  
    public String name;  
    public Integer age;  
    public UserModel(User entity) {  
        this.name = entity.name;  
        this.age = entity.age;  
    }  
}
```

Entity Class

```
@Entity
public class User {
    @Id
    public String name;
    public Integer age;
    public String password;
}
```